BASICS OF GenAI

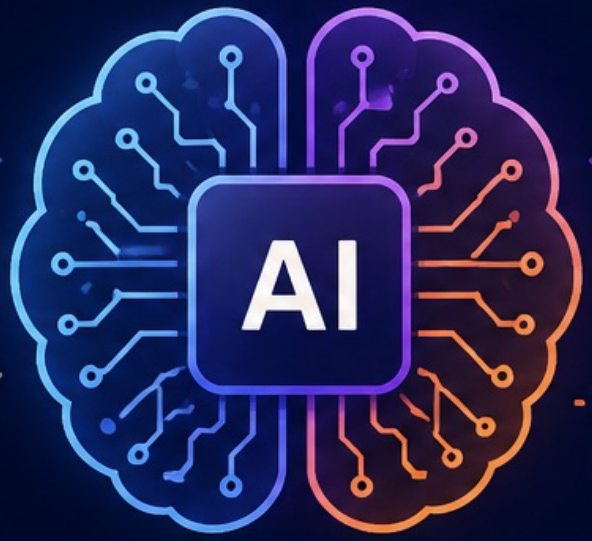
Understand. Create. Innovate.



TEXT GENERATION



IMAGE GENERATION



CODE GENERATION



IDEAS TO IMPACT





UNDERSTAND



CREATE



INNOVATE



EMPOWER

Lecture: Understanding Foundational Models

Agenda

1. Retrieval-Augmented Generation (RAG)
2. Vector Databases & Embeddings
3. LLM Evaluation
4. AI Agents & Agentic Workflows
5. Understanding GPT
6. ROPE
7. VIT

RAG

Reference: Human (LLM) Open Book Exam

KB

What? (External Knowledge base to your LLM (Brain))

Why? Improved Context, Up to date knowledge, reduce hallucination

No retraining of LLM, better retrieval

KB

↓

Vector DB

Why Vector Databases?

Traditional databases search by **exact match** (WHERE name = 'John').
Vector databases search by **similarity** (find items most similar to this embedding).
This enables:

Semantic search
"affordable laptops" matches "budget notebooks"

RAG pipelines
Retrieve relevant context for LLM generation

Recommendation systems
Find similar products, content, users

Anomaly detection
Identify outliers in embedding space

How They Work

1 INDEXING



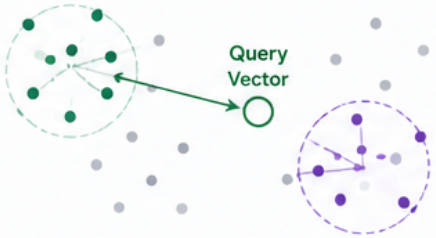
2 QUERYING



Distance Metrics

Metric	Formula	Best For
Cosine Similarity	$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$	Text embeddings (normalized)
Euclidean (L2)	$\sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$	Image embeddings
Dot Product	$\sum_{i=1}^{n} (a_i \times b_i)$	When magnitude matters

Visual Intuition



Points closer to the query vector are more similar.

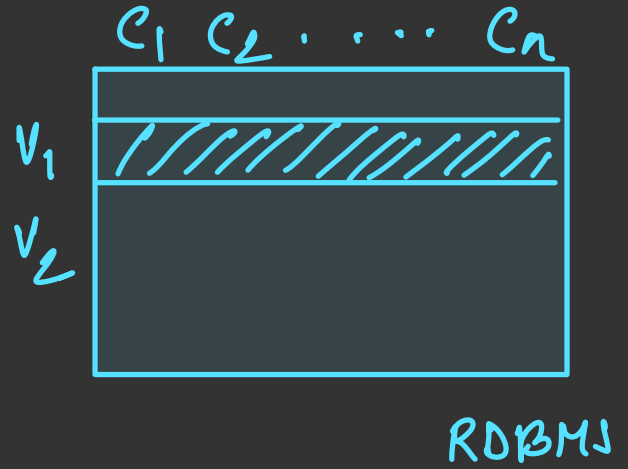
VDB
↳ super fast

By default → locally ✓

Vector Databases

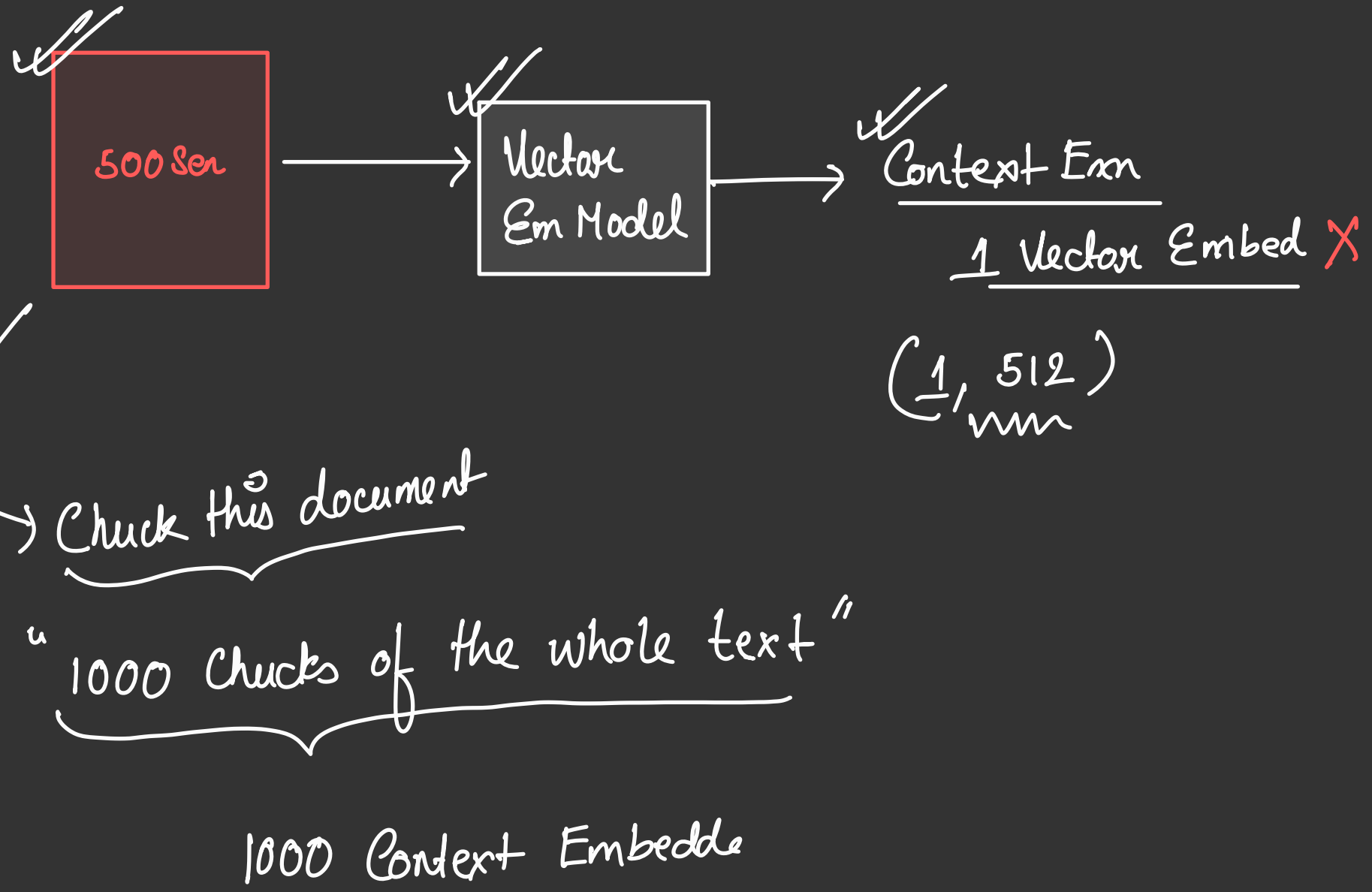
takes care of all these things.

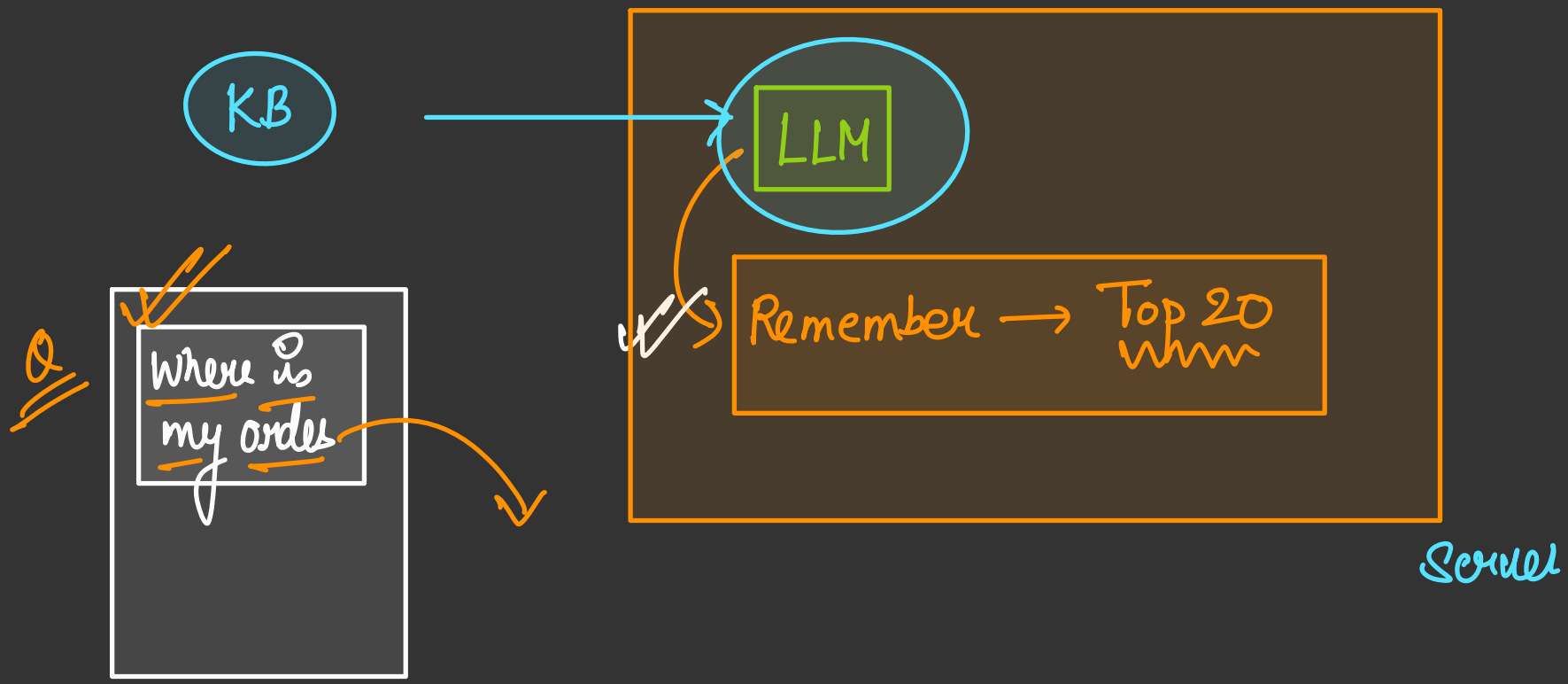
{ ANN,
HFSW,



FAISS, PineCone, Qvadrt

↳ Meta


500 Sen
Vector Em Model
Context Exn
1 Vector Embed X
(1, 512)
Chuck this document
" 1000 Chucks of the whole text "
1000 Context Embedde



AWS Bedrock

You C Vs Anth Vs Ama → NDA

LLM Evaluation with RAGAS

RAGAS helps you evaluate the quality of LLM responses using multiple metrics like **Faithfulness** and **Response Relevancy**.

Typical Evaluation Flow

1 Wrap LLM

Use an LLM (e.g., OpenAI GPT-4o-mini) wrapped for RAGAS evaluation.

2 Create Sample

Provide the user question, retrieved contexts, and the LLM response.

3 Choose Metric

Select the metric to evaluate (e.g., Faithfulness).

4 Run Evaluation

RAGAS evaluates the response using the chosen metric.

5 Get Score

Receive a score (0 to 1) indicating the quality of the response.

What is Evaluated?

User Question (e.g., query) + **Retrieved Contexts** (From Vector DB) + **LLM Response** (Answer) → **Metric Score** (0 to 1)

Common Metrics in RAGAS

Faithfulness

Measures how factually consistent the response is with the retrieved contexts.

High score = Response does not contain facts that are not in the context.

Response Relevancy

Measures how relevant the response is to the user question.

High score = Response addresses the question well.

Other Useful Metrics

Context Precision, Context Recall, Answer Correctness, etc.

Score Interpretation



0.0	0.5	1.0
Low	**Medium**	**High**
Needs Improvement	Acceptable	Excellent

Why It Matters?

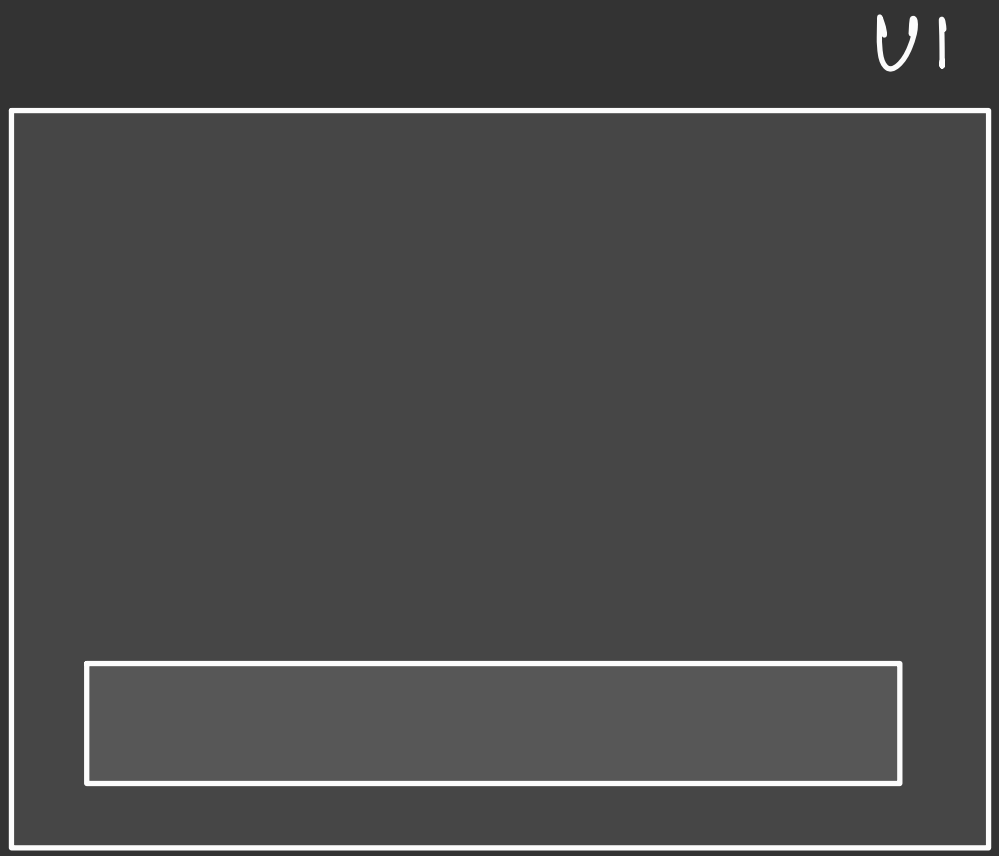
Systematic evaluation helps you build reliable RAG applications by ensuring responses are accurate, grounded in context, and truly relevant to user needs.

Q_1: Are you having fun in this module?

Q_2: Are you having a good time learning?

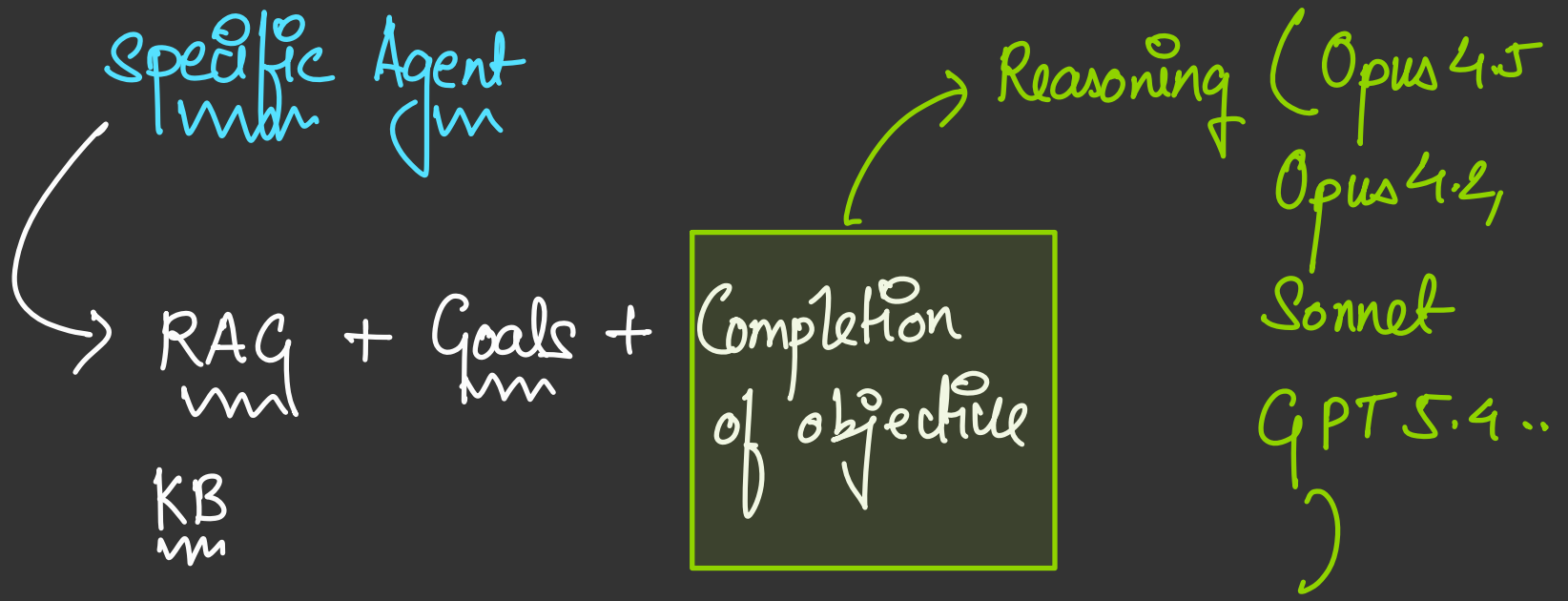
Agents

1) Understand a goal
2) Decides what actions to take
3) Uses external tools if needed
4) Observe & study the result
5) Continue till the task is completed.



Agent

goal


Specific Agent
RAG + Goals +
KB
Completion of objective
Reasoning (Opus 4.5
Opus 4.6
Sonnet
GPT 5.4 ..
)

Recap

RAG
- Framework to connect to KB

Vector DB
- FAISS

LLM Evaluation
- Ragas
  - Faithfulness
  - Relevancy

Agents
- Definition

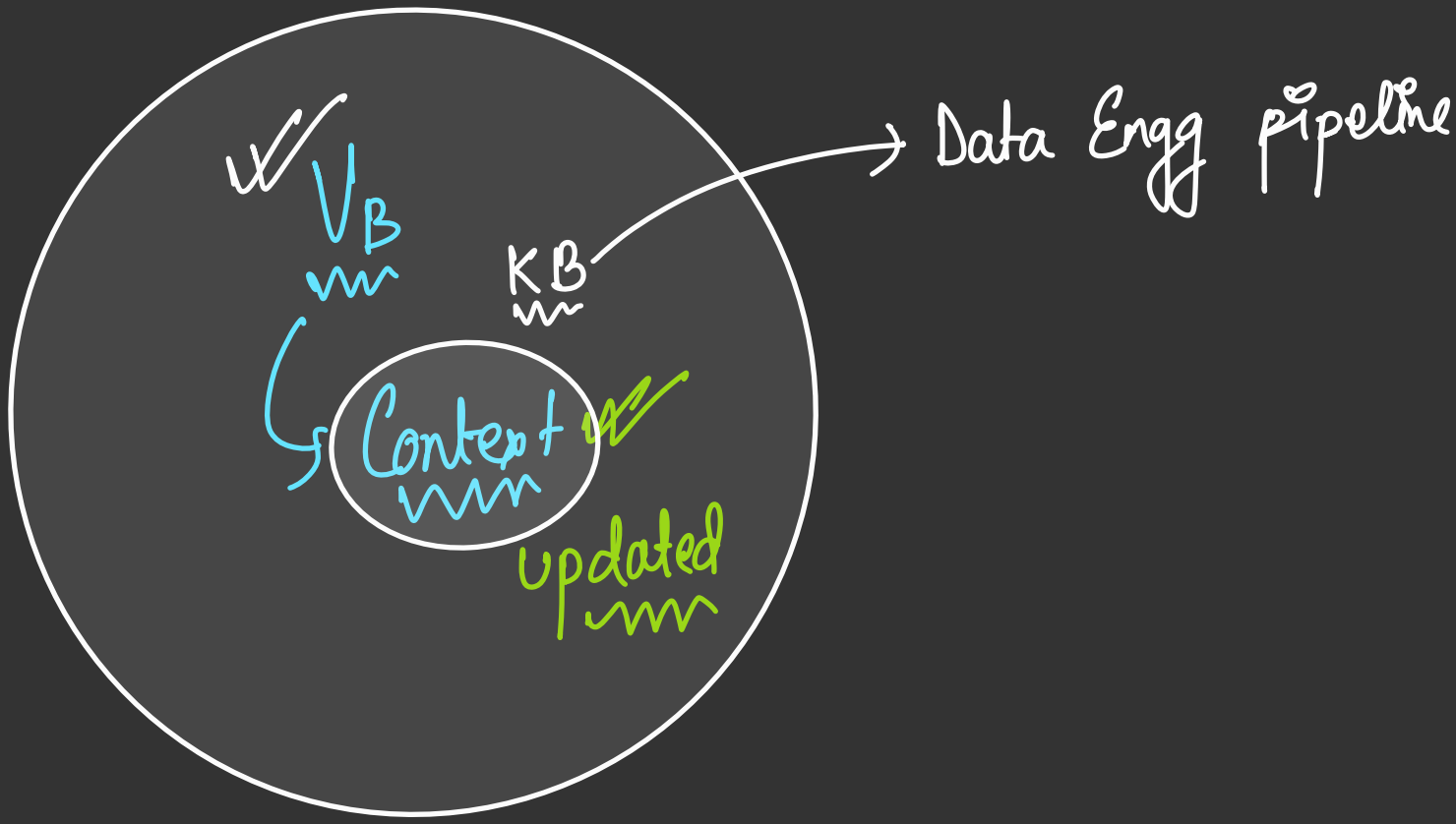
LLM
- GPT

ViT (GPT ⟵ Embedding)

Document 1 Vs Document 2

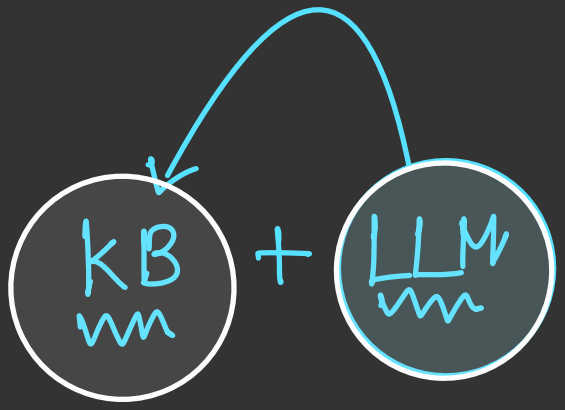
60%

overlapp


VB
KB
Context
updated
Data Engg pipeline

RAG

Concept





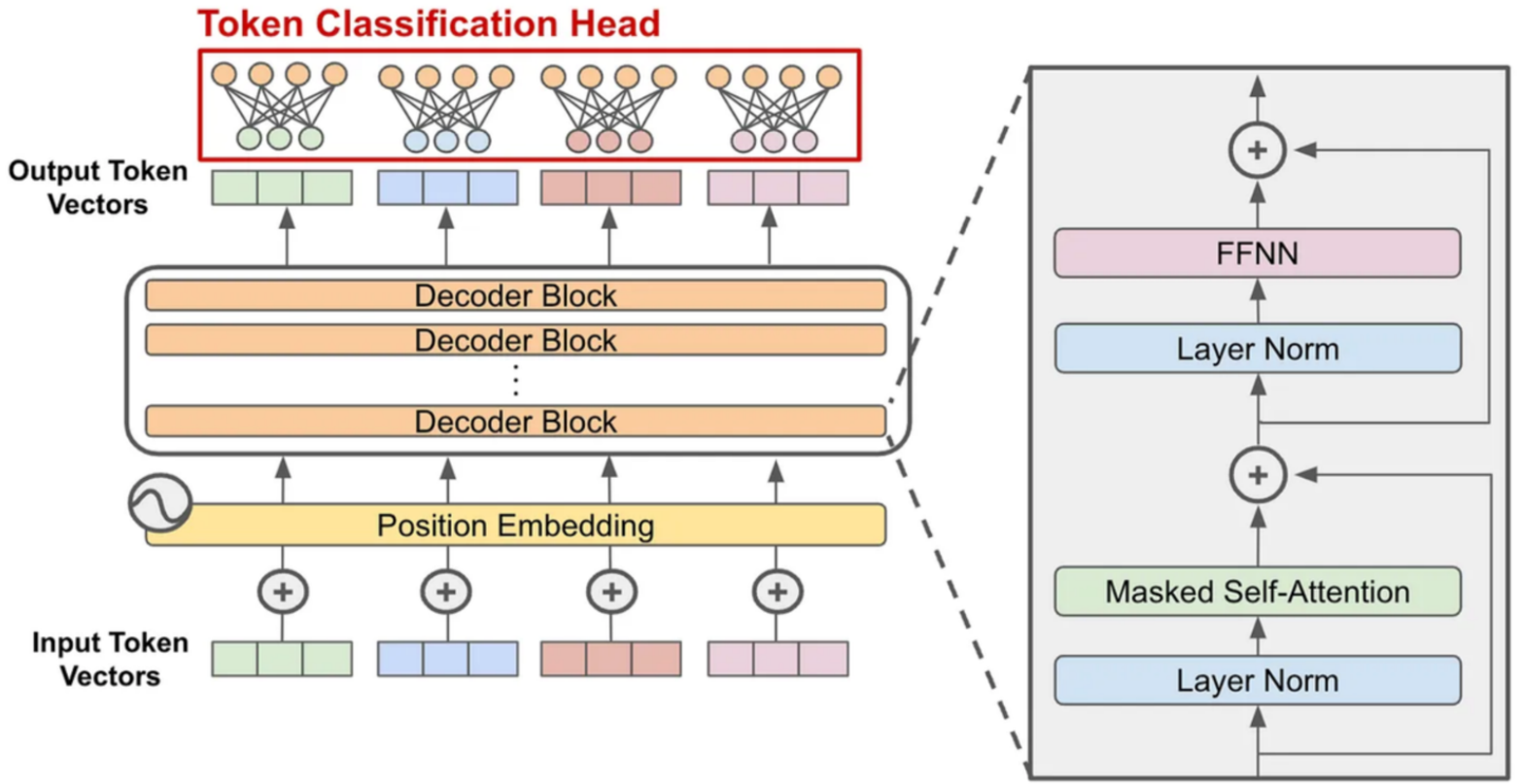
ML Model (Test Evaluation)

↳ Inference

AI Eng

↳ LLM (Response)

GPT (Decoder Only)



GPT ⟶ generates token one at a time.

How are you?

chat bots, text summarization, Auto complete

1.1 Decoder-Only Architecture Fundamentals

Core principle: Stack transformer decoder blocks that process sequences left-to-right with causal attention.

Why Decoder-Only Excels at Generation

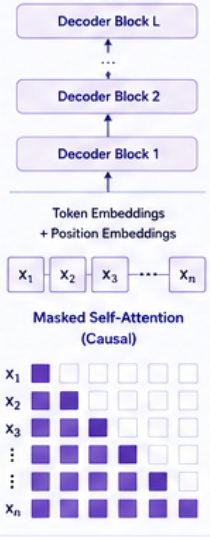
Training-inference alignment
Learns $P(x_i|x_1 \ldots x_{i-1})$ during training—exactly what's needed at generation time.

Unified processing
Prompt and generated text flow through identical attention mechanisms.

Scaling efficiency
Simpler architecture scales more predictably with compute and parameters.

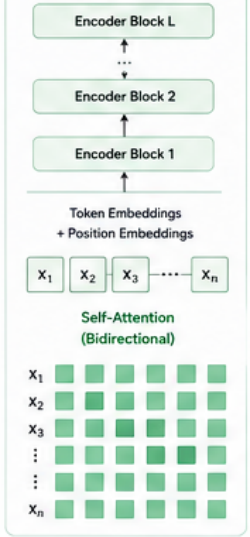
GPT-2 (Decoder-Only)

Autoregressive Generation (Left-to-Right)



BERT (Encoder-Only)

Bidirectional Understanding (All-to-All)



1.2 Data Flow Through the Model

How a token moves from input to next-token prediction

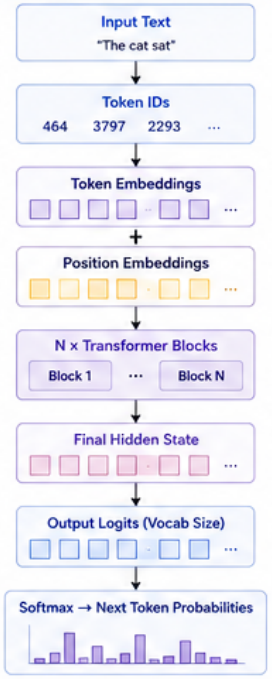
1 Tokenization (BPE)
Text → integers using BPE.
GPT-2: 50,257 tokens
Llama 3.1: 128K tokens

2 Embedding Lookup
Each token ID retrieves a vector from embedding matrix.
GPT-2 small: [50,257 × 768]

3 Position Information
Add position info to token embeddings.
GPT-2: Learned absolute positions [1024 × 768]
Llama: RoPE (Rotary Position Embeddings)

4 Transformer Block Processing (× N layers)
Each block: Pre-LN → Masked Self-Attention → Residual → Pre-LN → FFN → Residual

5 Output Projection
Linear projection to vocab size → Softmax → Next-token probabilities



1.3 Self-Attention Mechanism

Each token dynamically weights information from all other tokens.

Query (Q)
"What am I looking for?"

Key (K)
"What do I contain?"

Value (V)
"What information do I provide?"

Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Step-by-step

1. **QK^T computation**
   Each query dot-products with all keys → $[n \times n]$ score matrix
2. **Scaling**
   Divide by $\sqrt{d_k}$ to prevent softmax saturation ($d_k = 64$ for GPT-2)
3. **Softmax**
   Convert scores to probability distribution per row
4. **Value weighting**
   Multiply attention weights by values to get context-aware representations

Technical Specs (GPT-2 Small)

- Embedding dimension: 768
- Attention heads: 12
- Head dimension: 64 (768 / 12)
- QKV projection size: 2304 (768 × 3)

Why scaling matters?
Without $\sqrt{d_k}$ scaling, dot products grow large with dimension, pushing softmax into regions with near-zero gradients.

1.4 Causal (Masked) Self-Attention

Prevents tokens from attending to future positions.

Causal Mask (Triangular)

	1	2	3	4	...	n
1	0	−∞	−∞	−∞	...	−∞
2	0	0	−∞	−∞	...	−∞
3	0	0	0	−∞	...	−∞
4	0	0	0	0	...	−∞
⋮						
n	0	0	0	0	...	0

How it works

- $M_{i,j} = 0$ if $j \leq i$
- $M_{i,j} = -\infty$ if $j > i$
- Add M to attention scores before softmax.
- Positions where $j > i$ become $-\infty \rightarrow$ softmax $= 0$.
- Result: token at position i can attend only to tokens $1 \ldots i$.

Practical Implementation
Use `torch.triu()` to create upper-triangular mask and set those values to -inf or -1e9 (as in GPT-2).

Contrast with BERT
BERT uses bidirectional attention (no mask). Each token sees entire sequence → good for understanding, but not for generation.

1.5 Multi-Head Attention

Multiple attention operations in parallel to learn diverse patterns.

How it works

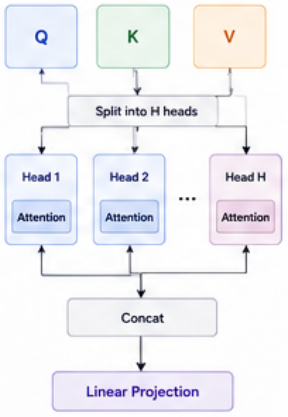
1 Split
Q, K, V from [batch, seq, d_model] to [batch, heads, seq, d_head]

2 Attend
Each head computes attention independently

3 Concatenate
Head outputs are concatenated back to d_{model} dimension

4 Project
Final linear projection mixes information across heads

Multi-Head Attention (Example)



GPT-2 Configuration

- 12 attention heads in parallel
- Each head: 64 dims (768 / 12)
- Different heads learn different aspects: syntax, semantics, positional patterns

Llama Innovation — GQA

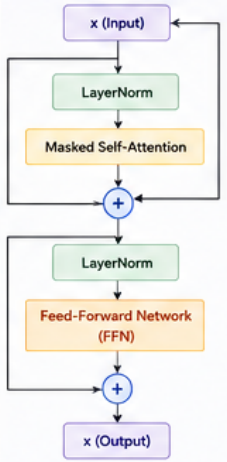
Grouped Query Attention
Multiple query heads share K, V projections

- Llama 3.1 8B: 32 query heads, 8 KV heads
- 75% reduction in KV cache memory
- Minimal accuracy degradation

1.6 The Complete Transformer Block

Combines attention and feed-forward processing with residual connections and normalization.

GPT-2 Block (Pre-LN)



Feed-Forward Network (FFN)

Two linear layers with activation between

Linear (768 → 3072) → GELU/SwiGLU → Linear (3072 → 768)

Expand 4× | Activation | Project back

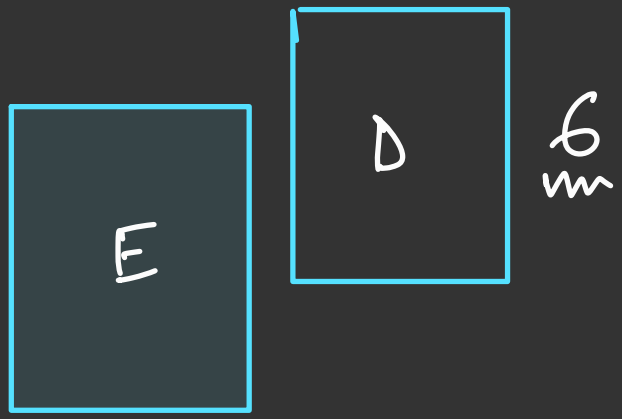
Why Pre-Norm Matters

- ✓ Normalizes BEFORE each sublayer (vs. post-norm)
- ✓ Training stability: smoother gradient flow
- ✓ Faster convergence: larger LR, skip warmup
- ✓ Used in modern models: GPT-2 onwards, Llama

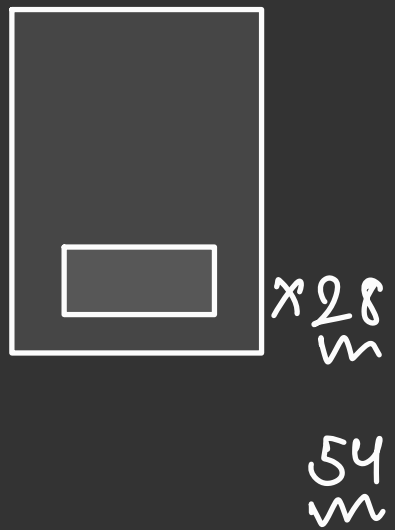
Residual Connections

Identity pathways allow gradients to flow directly through the network.
Essential for training deep models (48 layers in GPT-2 XL, 32+ in Llama).

Transformer



GPT

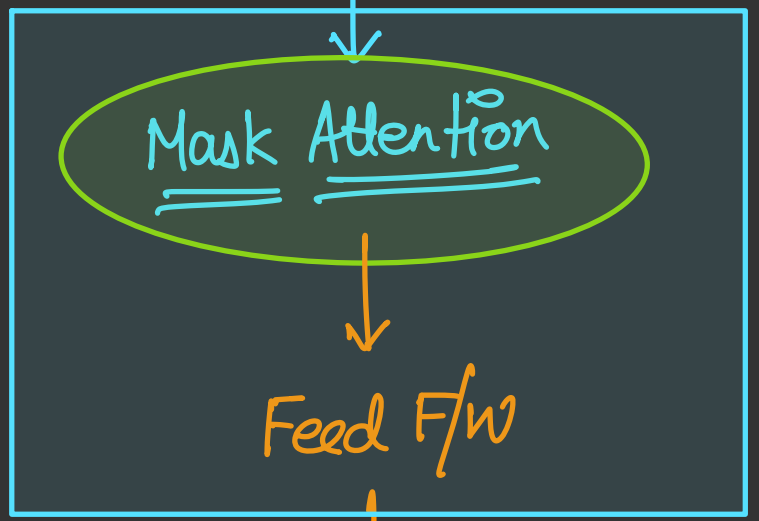


#GPT

L

How are you doing ? What is R → label is provided ?

→ Token → embedding + posi emb



How — — —

How are — —

How are you

. . .

. . .

Softmax

→ Loss : CCE

RoPE and the Long Context Revolution

How Rotary Position Embeddings unlocked the era of 100K+ context

The Context Length Challenge

Early transformers were limited to short contexts:

- Original transformer : 512 tokens
- BERT : 512 tokens
- GPT-2 : 1,024 tokens
- GPT-3 : 2,048 tokens

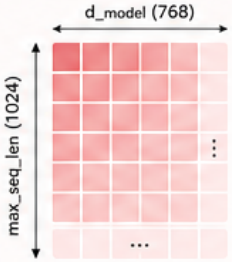
Why was extension hard? The problem was positional encoding.

Absolute Position Embeddings (GPT-2 style)

```
# Learned embedding for each position
position_embeddings = nn.Embedding(
    max_positions=1024, embedding_dim=768
)
```

Limitations

- **No extrapolation:** Can't handle sequences longer than training length
- **Memory cost:** Full embedding matrix [max_seq_len × d_model]
- **Position-specific:** Position 100 has no relation to position 101



The RoPE Breakthrough (2021)

Rotary Position Embedding (RoPE) revolutionized how transformers handle position information. Instead of adding position embeddings, RoPE rotates the query and key vectors.

$$f_q(x_m, m) = R_{\Theta,m} \cdot W_q x_m$$

Where $R_{\Theta,m}$ is a rotation matrix depending on position m.

Key insight: When you rotate Q at position m and K at position n, their dot product naturally encodes relative distance $(m - n)$.

$$q_m^T k_n = f_q(x_m, m)^T f_k(x_n, n) \propto \text{relative position encoding } (m - n)$$

Why RoPE enabled long context

1. **Relative position encoding**
   Distance between tokens matters, not absolute position.
2. **Length extrapolation**
   Can generalize to sequences longer than training (with techniques like YaRN, ABF).
3. **No parameter overhead**
   No learned embedding matrix, just rotation calculations.
4. **Decay properties**
   Attention naturally decays with distance.

The Long Context Timeline

Model	Year	Position Encoding	Context Length
GPT-2	2019	Learned absolute	1,024
GPT-3	2020	Learned absolute	2,048
Llama 1	2023	RoPE	2,048
Llama 2	2023	RoPE	4,096
GPT-4	2023	Unknown (likely RoPE variant)	32,768 → 128,000
Llama 3.1	**2024**	**RoPE**	**128,000**
Claude 3.5 Sonnet	2024	Proprietary	200,000

Llama 3.1's Context Achievement

From the technical report: Llama 3.1 achieves 128K context through:

- RoPE base frequency adjustment
- Continued pretraining on longer sequences
- Iterative scaling: 8K → 128K in stages

128K Context Achieved!

Why 128K Context Matters

Entire Codebases
Load and reason over full repositories, thousands of files.

Full Books for Analysis
Ingest and analyze entire books, papers, and long documents at once.

Long Conversation Histories
Maintain rich, extended dialogue context without losing important details.

Complex Reasoning
Perform multi-step reasoning over extensive information with higher accuracy.

The Impact
From short snippets to entire knowledge worlds—unlocking a new era of AI capability.

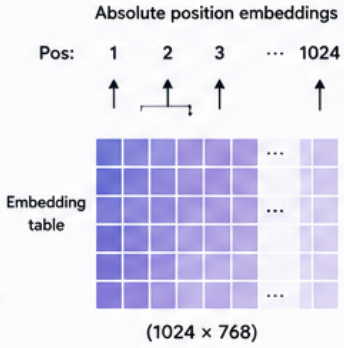
RoPE (Rotary Position Embeddings) Explained

Adding position to attention by rotation, not addition

1 The Problem with Absolute Position Embeddings

Earlier models (like GPT-2) add a learned vector for each position.

```
# Learned embedding for each position
position_embeddings = nn.Embedding(
    max_positions=1024,
    embedding_dim=768
)
```



- **No extrapolation:** Can't handle sequences longer than training length
- **Memory cost:** Full matrix [max_seq_len × d_{model}]
- **Position-specific:** Position 100 has no relation to position 101

2 What is RoPE?

RoPE (Rotary Position Embedding) adds position by rotating the Query and Key vectors in a multi-dimensional space.

$$f_q(x_m, m) = R_{\Theta,m} \cdot W_q x_m$$

Where $R_{\Theta,m}$ is a rotation matrix that depends on position m.

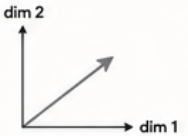
Key Idea

When Q at position m and K at position n are rotated, their dot product depends only on their relative distance $(m - n)$, not on absolute positions.

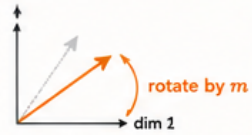
$$q_m^T k_n = f_q(x_m, m)^T f_k(x_n, n) \propto \text{relative position encoding } (m - n)$$

3 How RoPE Works (Simple View)

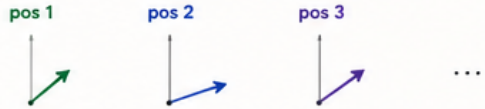
1. Represent each token (Q or K) as a vector.



2. For position m, rotate the vector by an angle that depends on m.



3. Different positions = different rotation angles.



4. Dot product after rotation naturally captures the relative distance between positions.

4 Why RoPE Enables Long Context

Relative position encoding
Model cares about distance between tokens, not their absolute position.

Length extrapolation
Can generalize to much longer sequences (with techniques like YaRN, ABF).

No parameter overhead
No learned position embedding matrix. Uses rotation math (no extra parameters).

Decay properties
Attention naturally decreases as distance between tokens increases.

5 The Long Context Timeline

Model	Year	Position Encoding	Context Length
GPT-2	2019	Learned absolute	1,024
GPT-3	2020	Learned absolute	2,048
Llama 1	2023	RoPE	2,048
Llama 2	2023	RoPE	4,096
GPT-4	2023	Unknown (likely RoPE variant)	32,768 → 128,000
Llama 3.1	**2024**	**RoPE**	**128,000**
Claude 3.5 Sonnet	2024	Proprietary	200,000

6 Llama 3.1's Context Achievement

From the technical report: Llama 3.1 achieves 128K context through:

- RoPE base frequency adjustment
- Continued pretraining on longer sequences
- Iterative scaling: 8K → 128K in stages

128K Context Achieved!

Why 128K Context Matters

- Entire codebases in context
- Full books for analysis
- Long conversation histories
- Complex reasoning over long documents

In One Line

RoPE encodes position by rotating vectors in space so that the model naturally focuses on how far apart tokens are—enabling much longer contexts without extra parameters.

Think of it like a clock hand



Absolute embeddings: Each hour on the clock is unrelated to others.



RoPE: What matters is the angle (distance) between two hands, not the exact hour.

Result

Better generalization to long sequences and a big leap in context length!

RoPE (Rotary Position Embeddings) – In Simple Terms

Think of RoPE like giving every word a slightly different rotation angle based on its position.

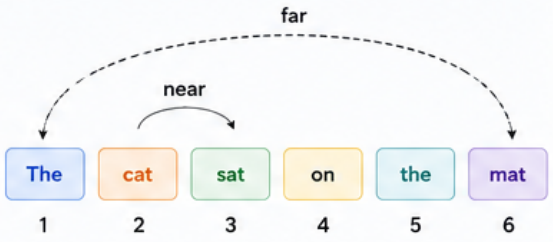
1 The Main Problem

Transformers need to understand:

- Which word came first
- Which word came later
- How far apart two words are

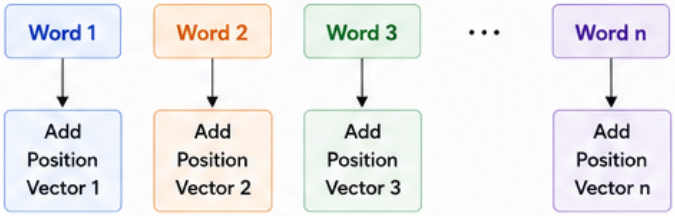
Example: *"The cat sat on the mat"*

- "cat" is near "sat"
- "The" is far from "mat"



2 Older Method: Absolute Position Embeddings

Older models like GPT-2 simply added a fixed position number.



Problems

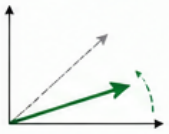
- The model memorizes exact positions
- Position 100 and 101 are treated completely separately
- Hard to generalize to longer text

So if the model trained on 1,024 tokens → It struggles beyond 1,024

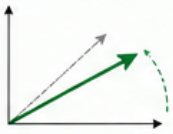
3 What RoPE Does Differently

Instead of adding position information... RoPE **rotates** the vectors.
Imagine every word embedding is an arrow.

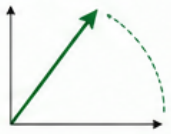
Position 1
Arrow rotated slightly.



Position 2
Arrow rotated a little more.



Position 50
Arrow rotated even more.



Key Idea

RoPE encodes position by rotation.

Different positions = different rotation angles.

4 Why Rotation Helps

Model does not care about:

- Exact position

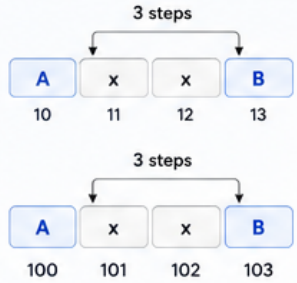
Model cares about:

- Relative distance between words

Example:

- Word A and Word B are 3 steps apart
- That relationship stays meaningful anywhere in the sentence

This is the magic.



5 Simple Analogy — Clock Hands

Older embeddings:
Every clock time is memorized separately



RoPE: What matters is the angle difference between clock hands



So:

- 1 → 2
- 50 → 51

Both represent "one step apart."

6 Why RoPE Enabled Long Context

Because rotations follow mathematical patterns, the model can:

- Extend to much longer sequences
- Understand relative positions naturally
- Reuse the same rotational logic at larger lengths



That's why modern models can handle:

32K	128K	200K+
context	context	context

7 In One Line

RoPE teaches transformers position by **rotating vectors**, so the model learns **how far words are from each other** instead of memorizing exact positions.

Vision Models: From CNNs to Transformers

The Evolution of Computer Vision

CNNs
2012–2020

Transformers
2020–Today

Computer vision has undergone a remarkable transformation in recent years. For over a decade, Convolutional Neural Networks (CNNs) dominated the field, achieving breakthrough results in image classification, object detection, and segmentation.

However, since 2020, a new paradigm has emerged: Vision Transformers (ViT).

Today, the majority of state-of-the-art vision models are transformer-based, marking one of the most significant shifts in computer vision history.

The CNN Era (2012–2020)

The Rise of CNNs

The modern deep learning revolution in computer vision began with AlexNet in 2012, which won the ImageNet competition with unprecedented accuracy. This sparked an explosion of CNN architectures:

AlexNet (2012)	Started the deep learning revolution
VGG (2014)	Showed deeper networks work better
ResNet (2015)	Introduced skip connections, enabling 152+ layers
Inception (2014)	Multi-scale feature extraction
EfficientNet (2019)	Optimized architecture scaling

Impact of CNNs

- ✓ Massive accuracy improvements
- ✓ Enabled real-world applications
- ✓ Scaled to billions of parameters
- ✓ Dominated the field for nearly a decade

CNNs achieved state-of-the-art results across many vision tasks!

How CNNs Work

CNNs process images through a hierarchy of convolutional layers:



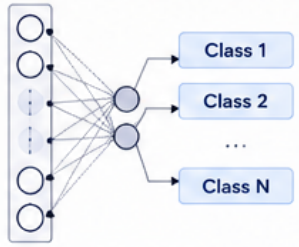
Input Image
(H × W × 3)

1. Convolution
Apply filters to detect local patterns (edges, textures, corners)

2. Activation
ReLU introduces non-linearity to learn complex patterns

3. Pooling
Downsample feature maps (e.g., Max Pooling) to reduce spatial size

4. Stacked Layers
Repeat conv + activation + pooling to learn high-level representations



5. Classification Head
Flatten features and pass through fully connected layers to predict class probabilities

Local Connectivity
Each filter looks at a small neighborhood

Weight Sharing
Same filter used across the image

Translation Equivariance
Features can be detected anywhere in the image

Hierarchical Features
From low-level edges to high-level concepts

Vision Transformers (ViT)

From CNNs to Transformers: A New Paradigm in Computer Vision

The Birth of Vision Transformers

In October 2020, Google Research published **"An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale" (Dosovitskiy et al.)**, introducing Vision Transformers (ViT).

The Revolutionary Idea:

> "Treat an image as a sequence of patches, just like words in a sentence"

This simple but profound insight showed that transformers could match or exceed CNN performance on image tasks.

The Evolution of Computer Vision

CNN Era (2012–2020)

CNNs dominated the field for over a decade, achieving breakthroughs in image classification, object detection, and segmentation.

Transformer Era (2020–Today)

A new paradigm emerged. Today, majority of state-of-the-art vision models are transformer-based.

One of the most significant shifts in computer vision history!

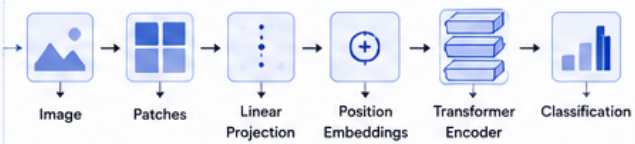
Introduction to Vision Models

Vision models have evolved significantly from CNNs to Transformers. Vision Transformers (ViT) apply the transformer architecture—originally designed for NLP—to computer vision tasks by treating images as sequences of patches.

Key Innovation:

Instead of processing images pixel-by-pixel, ViT splits images into fixed-size patches, linearly embeds them, and processes them through transformer encoder blocks.

Architecture Overview



ViT vs CNN

CNN

Processes images through convolutional filters, building hierarchical features.

VS

ViT

Splits image into patches, treats them as tokens, applies self-attention (Transformer).

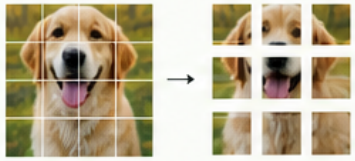
3 Vision Transformer (ViT) – Simple Step-by-Step Guide

Simple Idea: Treat an image like a sentence

Words in NLP → Patches in Vision

Step 1
Split Image → Patches (like words)

Divide the image into non-overlapping fixed-size patches (e.g., 16×16 pixels).



Example: 224×224 image → 14×14 = 196 patches (plus 1 CLS token later)

Step 2
Patches → Embeddings (vectorize them)

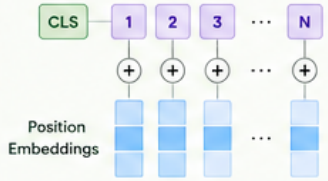
Each patch is flattened and passed through a linear layer to create an embedding (vector representation).



Patch (16×16×3) → Vector (e.g., 768-dim)

Step 3
Add Special Tokens (CLS + Position info)

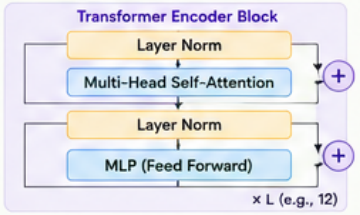
Add a learnable [CLS] token at the beginning and position embeddings to retain positional information.



N = number of patches (e.g., 196)

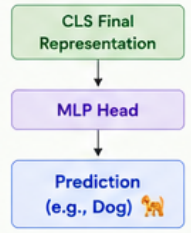
Step 4
Process through Transformer (12 layers)

The sequence is fed into a stack of Transformer Encoder blocks (e.g., 12 layers). Self-attention lets each patch interact with all other patches.



Step 5
Classify using (CLS token

The final representation of the [CLS] token is used for classification.



Why ViT Works?

Global View: Self-attention sees the entire image at once.

Scalable: Performance improves with more data & compute.

Flexible: Works well for various tasks – classification, detection, segmentation.

Unified Architecture: Same transformer backbone can be adapted to many tasks.